

**Company: Amazon**

**Interview Type: CASE\_STUDY**

-----

## Q1: Real-time Trending Products in Sliding Windows

**Difficulty:** Hard

**Tags:** Data Structures, Sliding Window, Hash Map, Doubly Linked List, Frequency Map

**Problem Statement:** Amazon Live needs a system to track the top  $K$  most viewed products within a rolling 60-minute window. The system receives a high-volume stream of `view(productId, timestamp)` events. Implement a data structure that supports `recordView(productId, timestamp)` and `getTopTrending()` which returns the  $K$  most viewed products in the last 60 minutes.

**Expected Approach:** Use a Hash Map to store `productId` and its total count. Use a queue or a sliding window approach to remove expired timestamps. This requires iterating through the map to find the top  $K$ , leading to  $O(N \log K)$  or  $O(N)$  for every query.

**Optimized Approach:** Use a combination of a Hash Map (for  $O(1)$  lookups), a Doubly Linked List of "Frequency Buckets" (similar to LFU Cache), and a sliding window of events. Each bucket represents a frequency count and contains a set of product IDs. As a product is viewed, it moves to a higher frequency bucket. A background process or a lazy-cleanup mechanism removes events older than 60 minutes, decrementing product frequencies and moving them to lower buckets.

**Time Complexity:** `recordView`:  $O(1)$  amortized; `getTopTrending`:  $O(K)$ .

**Space Complexity:**  $O(E + P)$ , where  $E$  is the number of events in the window and  $P$  is the number of unique products.

**Optimizations:** Use a time-bucketed approach (e.g., 1-minute granularity) to batch expired events instead of processing them second-by-second to reduce overhead.

**Hints:** Think about how LFU (Least Frequently Used) cache works. How can we adapt it to handle deletions when a timestamp expires?

-----

## Q2: Distributed Warehouse Inventory Balancing

**Difficulty:** Hard

**Tags:** Graph Theory, Tree DP, Centroid Decomposition, Optimization

**Problem Statement:** Amazon's fulfillment network is modeled as a tree of  $N$  warehouses connected by  $N-1$  logistics routes. Each warehouse  $i$  has a current inventory  $C_i$  and a target inventory  $T_i$ . You need to find the "Balance Center"—a warehouse location such that if all inventory redistributions are coordinated through this node, the maximum distance any single unit of stock travels is minimized.

**Expected Approach:** For every node, calculate the distance to all other nodes and simulate the inventory flow. This results in  $O(N^2)$  complexity, which is too slow for large logistics networks.

**Optimized Approach:** This is a variation of the Tree Centroid problem. Use Dynamic Programming on Trees. First, calculate the net surplus or deficit for every subtree. Use a two-pass DFS (rerooting technique) to calculate the "weighted eccentricity" for each node, where weights are the inventory units moved. The node that minimizes the maximum path cost of an inventory unit is the optimal center.

**Time Complexity:**  $O(N)$ .

**Space Complexity:**  $O(N)$  for tree adjacency list and DP arrays.

**Optimizations:** Use Bit Manipulation if flags are needed for warehouse types. Ensure the recursion depth is handled or use an iterative DFS for very deep trees.

**Hints:** If you move a unit from node A to node B, it passes through the unique path in the tree. Focus on the node that splits the tree into components with the most balanced "net inventory flow."

-----

### Q3: Critical Warehouse Infrastructure

**Difficulty:** Hard

**Tags:** Graph Theory, DFS, Articulation Points, Supply Chain

**Problem Statement:** Amazon's logistics network is represented as a graph where nodes are fulfillment centers and edges are high-speed transport routes. To ensure 1-day delivery, the network must remain connected. A "Critical Center" is defined as a fulfillment center that, if it goes offline (due to maintenance or power failure), would result in at least one other center becoming unreachable from the rest of the network. Given an adjacency list of  $V$  centers and  $E$  routes, identify all Critical Centers to prioritize for backup power systems.

**Expected Approach:** Use a modified Depth First Search (DFS) to find Articulation Points. Maintain `discovery\_time` and `low\_link\_value` arrays for each node. For each edge  $(u, v)$ , if  $low[v] \geq discovery[u]$ , node  $u$  is a candidate for being a critical center.

**Optimized Approach:** Implement Tarjan's Algorithm or the Hopcroft-Tarjan algorithm. Initialize a global timer. During DFS, update  $low[u]$  as the minimum of  $discovery[u]$ ,  $discovery[neighbor]$  (back-edge), and  $low[child]$  (tree-edge). Handle the root of the DFS tree as a special case (critical only if it has  $>1$  child).

**Time Complexity:**  $O(V + E)$

**Space Complexity:**  $O(V)$

**Optimizations:** Use an adjacency list instead of a matrix to handle sparse logistics networks efficiently. Use an iterative DFS to prevent stack overflow in extremely large geographic networks.

**Hints:** Think about what happens if a node is part of a cycle versus a bridge. Which one's removal breaks connectivity?

-----

### Q4: Hierarchical Inventory Aggregator

**Difficulty:** Medium-Hard

**Tags:** Tree, Recursion, Memoization, System Design

**Problem Statement:** Amazon's product catalog is organized into a deep hierarchy of categories (e.g., Electronics -> Computers -> Laptops). Each category node contains a count of unique items currently in stock. However, stock numbers are only updated at the leaf nodes (specific products). A "Category Overview" request requires the total sum of stock for a category and all its sub-categories. Given a tree structure where each node has an `inventory\_count`, and frequent updates occur at leaf nodes, design a system to handle "GetSum(category\_id)" and "Update(category\_id, new\_count)" calls efficiently.

**Expected Approach:** Use a bottom-up recursive traversal to pre-calculate sums. For updates, traverse from the leaf to the root, updating parent nodes.

**Optimized Approach:** Map the tree into a linear array using Segment Tree or Fenwick Tree (Binary Indexed Tree) logic via Euler Tour traversal (Entry/Exit times). Each category becomes a contiguous range  $[entry, exit]$ . Update operations become  $O(\log N)$  and range sum queries become  $O(\log N)$ .

**Time Complexity:**  $O(\log N)$  for both Update and Query.

**Space Complexity:**  $O(N)$  to store the tree and the BIT/Segment Tree.

**Optimizations:** If the tree is relatively static but updates are frequent, use a Fenwick Tree for lower constant factor overhead compared to a Segment Tree.

**Hints:** How can we represent a sub-tree as a range in a 1D array? Consider how "Entry Time" and "Exit Time" define a node's scope.

-----

## Q5: Optimal Delivery Drone Battery Management

**Difficulty:** Hard

**Tags:** Greedy, Priority Queue, Interval Scheduling

**Problem Statement:** Amazon is deploying drones for Prime Air. A drone must complete  $N$  deliveries, each with a required energy consumption  $E_i$  and a strict delivery window  $[S_i, T_i]$ . Drones can recharge at "Charging Hubs" instantly, but each hub has a limited number of charging slots  $K$ . Given a set of delivery tasks and hub availability, determine the minimum number of drones required to complete all deliveries without any drone running out of battery (assuming a max capacity  $C$ ) or missing a time window.

**Expected Approach:** Sort all delivery tasks by their start times. Use a min-priority queue to track the end times of currently active drones. For each new task, check if any drone in the queue has finished its previous task and has enough remaining battery to reach the next location.

**Optimized Approach:** Use a Greedy strategy with a Min-Heap. Sort deliveries by  $S_i$ . The heap stores drones based on their "available time." For each task, attempt to reuse a drone from the heap that finished earliest. If the drone's remaining charge  $C_{rem} < E_i$ , it must be routed to a charging hub first, adding a penalty time  $P$  to its availability. If no drone is available or can reach the destination in time after charging, instantiate a new drone.

**Time Complexity:**  $O(N \log N)$  due to sorting and  $N$  heap operations.

**Space Complexity:**  $O(N)$  to store drone states in the heap.

**Optimizations:** Use a secondary Fenwick tree to manage charging hub slot availability if hubs are geographically dispersed and have time-varying constraints.

**Hints:** Think of this as a variation of the "Meeting Rooms II" problem but with a stateful resource (battery) that requires periodic replenishment.

-----

## Q6: Real-time Detection of Seasonal Product Clusters

**Difficulty:** Hard

**Tags:** Graph, Union-Find, Sliding Window

**Problem Statement:** On Prime Day, Amazon needs to identify "viral clusters" of products. Two products are "linked" if they are purchased together by at least  $M$  users within a sliding 10-minute window. A cluster is a set of products where every product is reachable from any other via links. Given a stream of (User, Product, Timestamp) tuples, return the size of the largest viral cluster at any given moment.

**Expected Approach:** Use a Hash Map to track user-product pairs within the window. When a threshold  $M$  is met, create an edge in a graph. Periodically rebuild the graph to remove expired links.

**Optimized Approach:** Use a sliding window with a Disjoint Set Union (DSU) and a Frequency Map. Maintain a queue of transactions. As a transaction enters the window, increment the co-purchase count for the product pair. If count reaches  $M$ , perform a `union()` in the DSU. To handle departures from the window, use a DSU with Rollback (Persistence) or a "Link-Cut Tree" to dynamically remove edges without full re-computation. Track the `max_size` component within the DSU structure.

**Time Complexity:**  $O(E \log V)$  or  $O(E \alpha(V))$  depending on the DSU implementation, where  $E$  is the number of co-purchase events.

**Space Complexity:**  $O(P^2)$  in the worst case to store product pair frequencies, where  $P$  is the number of unique products.

**Optimizations:** Use a Bloom Filter to quickly discard pairs that haven't reached the threshold  $M$ , reducing the memory footprint of the frequency map.

**Hints:** Focus on how to efficiently "undo" a connection when the time window slides past the first user's co-purchase of that specific pair.

-----

## Q7: Optimized Warehouse Shipping Consolidation

**Difficulty:** Hard

**Tags:** Dynamic Programming, Bitmask, Combinatorics, Logistics

**Problem Statement:** Amazon needs to fulfill a customer order containing  $K$  unique items. There are  $N$  fulfillment centers (warehouses). Each warehouse  $i$  stocks a specific subset of the requested items (represented as a bitmask) and has a fixed shipping cost  $S_i$  associated with sending a package from that location. If multiple items are picked from the same warehouse, the shipping cost is incurred only once. Determine the minimum total shipping cost to fulfill the entire order of  $K$  items.

**Expected Approach:** A recursive approach with memoization exploring all subsets of warehouses. This results in  $2^N$  complexity, which is inefficient for large  $N$ .

**Optimized Approach:** Use Dynamic Programming with Bitmasking. Let  $dp[mask]$  be the minimum cost to fulfill the set of items represented by the bitmask. For each warehouse, iterate through all existing  $dp$  states and update the cost:  $dp[mask \mid warehouse\_items] = \min(dp[mask \mid warehouse\_items], dp[mask] + warehouse\_cost)$ .

**Time Complexity:**  $O(N \cdot 2^K)$ , where  $N$  is the number of warehouses and  $K$  is the number of items.

**Space Complexity:**  $O(2^K)$  to store the DP table.

**Optimizations:** Prune warehouses that provide no new items or have costs higher than a combination of others providing the same items. Use a bottom-up iterative DP to avoid recursion overhead.

**Hints:** Think of this as the Set Cover problem, but with small  $K$  (items) and larger  $N$  (warehouses). How can you represent the state of "items collected" efficiently?

-----

## Q8: Real-time Distributed "Heavy Hitter" Detection

**Difficulty:** Hard

**Tags:** Data Streams, Probabilistic Data Structures, System Design, AWS

**Problem Statement:** An AWS API Gateway receives millions of requests per second. You need to identify "Heavy Hitters"—IP addresses that contribute to more than 0.1% of the total traffic in a rolling 5-minute window—to trigger automated DDoS throttling. You have strictly limited memory (128MB) and cannot store every unique IP address.

**Expected Approach:** Use a Hash Map to count occurrences of every IP. This fails because the number of unique IPs can exceed memory limits during a peak or attack.

**Optimized Approach:** Implement a Count-Min Sketch (a probabilistic data structure) combined with a Min-Heap of size 1000 (to track the top 0.1%). For every incoming IP, increment its counters in the Count-Min Sketch using  $d$  different hash functions. Estimate the frequency by taking the minimum of these counters. If the estimate exceeds the threshold, update the Min-Heap.

**Time Complexity:**  $O(1)$  per request for updates and lookups.

**Space Complexity:**  $O(w \cdot d)$  for the Count-Min Sketch, where  $w$  is width and  $d$  is depth, independent of the number of unique elements.

**Optimizations:** Use a "Heavy Keeper" algorithm or "Space-Saving" algorithm to reduce the frequency of overestimation (false positives) inherent in Count-Min Sketch.

**Hints:** If you can't store everything, can you store a summary? How do you handle the "rolling window" aspect—perhaps by using multiple sketches for sub-windows and aggregating them?

-----

## Q9: Fulfillment Center "Stable Throughput" Analysis

**Difficulty:** Hard

**Tags:** Sliding Window, Monotonic Queue, Data Stream

**Problem Statement:** Amazon's fulfillment centers monitor the number of packages processed per minute. To optimize labor allocation, management needs to identify the longest continuous time

interval where the "throughput stability" is maintained. Stability is defined as an interval where the absolute difference between the maximum and minimum number of packages processed in any two minutes within that window does not exceed a threshold  $K$ . Given an array `throughput` of size  $N$  and an integer  $K$ , return the length of the longest such stable interval.

**Expected Approach:** Use a sliding window with two heaps (max-heap and min-heap) to track the maximum and minimum values within the current window. Expand the right pointer and, if the stability condition is violated, shrink the left pointer until the condition is met.

**Optimized Approach:** Use a sliding window combined with two Monotonic Deques. One deque maintains the indices of elements in decreasing order (to track the maximum), and the other maintains indices in increasing order (to track the minimum). This allows for  $O(1)$  access to the max and min of the current window and  $O(1)$  amortized updates for both pointers.

**Time Complexity:**  $O(N)$

**Space Complexity:**  $O(N)$

**Optimizations:** Use primitive arrays for deques instead of high-level collections in languages like Java to minimize overhead during high-frequency data ingestion.

**Hints:** Think about how to find the max and min of a sliding window in linear time. How does the window size change when the condition  $\max - \min \leq K$  is broken?

-----

## Q10: Amazon Fresh: Optimal Cold-Chain Routing

**Difficulty:** Hard

**Tags:** Graph, Dijkstra's Algorithm, State-Space Search

**Problem Statement:** An Amazon Fresh delivery van must deliver a temperature-sensitive package from a source warehouse  $S$  to a customer location  $D$  in a city represented as a directed graph of  $N$  locations and  $M$  roads. Each road has a travel time  $T$ . The package has a "freshness life" of  $L$  minutes. However, there are  $C$  locations in the city that act as "Re-icing Stations" where the package freshness can be fully restored to  $L$ . Find the minimum time to reach  $D$  from  $S$  without the package ever exceeding its freshness limit  $L$  between stations.

**Expected Approach:** Build a graph where each node is a location. Perform a Breadth-First Search (BFS) if all weights are equal, or Dijkstra if weights vary, but the state must only track the current location.

**Optimized Approach:** Use a modified Dijkstra's algorithm where the state is defined as `(current_location, remaining_freshness)`. Maintain a 2D `dist` array `min_time[location][remaining_freshness]` initialized to infinity. Use a Priority Queue to store `(time_elapsed, current_location, remaining_freshness)`. When arriving at a Re-icing Station, reset `remaining_freshness` to  $L$ . Prune paths where the `time_elapsed` is already greater than a previously recorded `min_time` for that specific location and freshness level.

**Time Complexity:**  $O((M + N) \cdot L \cdot \log(N \cdot L))$

**Space Complexity:**  $O(N \cdot L)$

**Optimizations:** If the number of Re-icing Stations is small, precompute all-pairs shortest paths between stations using Dijkstra and solve as a shorter graph problem.

**Hints:** The standard Dijkstra finds the shortest path based on one variable. How can you incorporate the "freshness" constraint into the graph state? Consider each location at each possible freshness

level as a distinct node in a virtual graph.

-----